



[REFERENCE IMPLEMENTATION]

AGENTIC AI PIPELINE · REAL-TIME SALES COACHING · PORTFOLIO

Sales Call Copilot

A four-agent pipeline that detects a live objection, grounds a suggestion in real enablement content, and scores its own output — reported with exactly where each stage does, and doesn't, hold up.

Anthony Chinedu Echem

Machine Learning Engineer & Data Scientist

Retrieval-augmented generation · deterministic rubric scoring · model-tier escalation

EXECUTIVE SUMMARY

A real win-rate signal, and three metrics that look stronger from a distance than up close

This project builds a 4-agent pipeline — detect the objection, retrieve grounded enablement content, generate a coaching suggestion, score the result — and evaluates each stage on its own terms. Reps who used the recommended play won 2.9× more often. Three other headline metrics needed a second look before they earned their place in this deck.



2.9×

Win-rate lift, recommended play vs. improvised response



87.5%

Semantic retrieval precision — clears the 85% target



74.4%

Objection-detection accuracy — a bimodal 100% / 0% split underneath



0.829

Rubric-vs-"human" correlation — against a simulated proxy, not real coaches



Bottom line

Every headline number in this deck ships with the evaluation design behind it — including the two that need a caveat to be read honestly (Sections 03 and 05).

Presentation agenda



01

The Business Problem

Why objection handling varies rep to rep, call to call



02

Data & Call Corpus Profile

1,200 calls, a 300-document knowledge base



03

Agent 1: Objection Detection

A rule-based classifier, and the split hiding in 74.4%



04

Agents 2-4: Retrieve, Generate, Score

TF-IDF grounding, tiered generation, deterministic rubric



05

Evaluation & Validation

Three studies, three honest reads on what they prove



06

Business Impact

A 2.9× win-rate lift, and what it can and can't claim



07

Deployment & Responsible Use

The API, a model card, and coaching-not-autonomy guardrails



SECTION ONE

The Business Problem

The same objection lands on every rep's calls — the response to it shouldn't depend on who happens to be talking.

Every rep improvises their own response, every time

Objection handling is one of the most-studied, best-documented parts of a sales playbook — and one of the least consistently applied in the room, especially under a live prospect's pushback.



Enablement content goes unused

Battlecards and case studies live in a shared drive — useful only if a rep remembers the right one exists, mid-call, under pressure.



New reps improvise the most

The reps with the least pattern-matched experience face the same objections as everyone else, with the least grounded material to draw on.



The gap has a real number attached

21.1% win rate on calls where the rep improvised, vs. 61.3% when they used the recommended play — the gap this project targets (Section 06).

The question this project asks: can a system listen alongside the rep, recognize the objection, and surface the right grounded material before the moment passes?

Coaching happens after the call, not during it

Most sales orgs rely on post-call review — valuable, but it can’t help a rep in the moment the objection actually lands.



Call happens, live

The objection lands in real time — whatever the rep says next, they say without a lookup or a coach in the room



Review happens later

A manager listens to the recording days later, if at all, and gives feedback the rep can use on the next call, not this one



Feedback is qualitative

Coaching notes rarely trace back to a specific piece of enablement content — hard to make consistent across a whole team



This project’s target

A structured suggestion, grounded in real content, in the moment it’s needed — feeding the rep, not replacing their judgment

44.3%

overall win rate,
this call corpus

This is the blended baseline across 1,200 calls — the number every subgroup comparison in this deck is measured against.

REFRAMING THE TASK

Four small jobs, chained, beat one big black box

Rather than one model trying to do everything, this pipeline splits the problem into four narrow, individually testable stages — each one auditable on its own.

Why split it into four agents?

detect → **retrieve** →
generate → **score**

Each stage has its own, independently checkable output — a classifier label, a ranked document list, a generated string, a numeric score. None of them hides inside a single opaque call.

What each stage is accountable for



Detection: is this objection classified correctly? (Section 03)



Retrieval: is the surfaced content actually relevant? (Section 04-05)



Generation + scoring: is the suggestion grounded, cited, and fast? (Section 04-05)

What this project set out to prove



Classify the objection in real time

Detect one of eight known objection types from a live transcript snippet, with no model latency to speak of.



Ground every suggestion in real content

Retrieve the actual battlecard or case study behind a coaching suggestion — never an unsourced claim.



Escalate only when it's worth it

Route routine objections to a fast, cheap model and complex ones to a stronger one — not the same tier for everything.



Score every call the same way, every time

Replace subjective post-call review with a deterministic, itemized rubric — removing rater drift.



Validate each stage on its own evidence

Test detection, retrieval, and scoring separately — a strong end-to-end demo isn't validation.



Connect the pipeline to a real outcome

Check whether using the recommended play actually correlates with winning the deal.



SECTION TWO

Data & Call Corpus Profile

1,200 calls, 300 pieces of enablement content, and one small gold set standing in for a real coach's judgment.

DATASET

A call corpus, a knowledge base, and a tiny gold set

The three data sources this pipeline touches: 1,200 labeled calls to test detection and outcomes against, 300 enablement documents to retrieve from, and 8 hand-labeled queries to validate the retriever.

Source files

call_transcripts.csv	1,200 calls — objection, stage, deal size, outcome	Corpus
knowledge_base.csv	300 battlecards & case studies, 8 objection types	KB
gold_eval_set.csv	8 hand-labeled queries, one per objection type	Gold
tfidf_index.joblib	fitted TF-IDF vectorizer + document matrix	Index
retrieval_eval_report.json	precision@3, exact and semantic	Eval
rubric_agreement_report.json	rubric-vs-simulated-human correlation	Eval

Corpus at a glance

1,200

calls across 4 pipeline stages

8

objection types

300

knowledge-base documents

44.3%

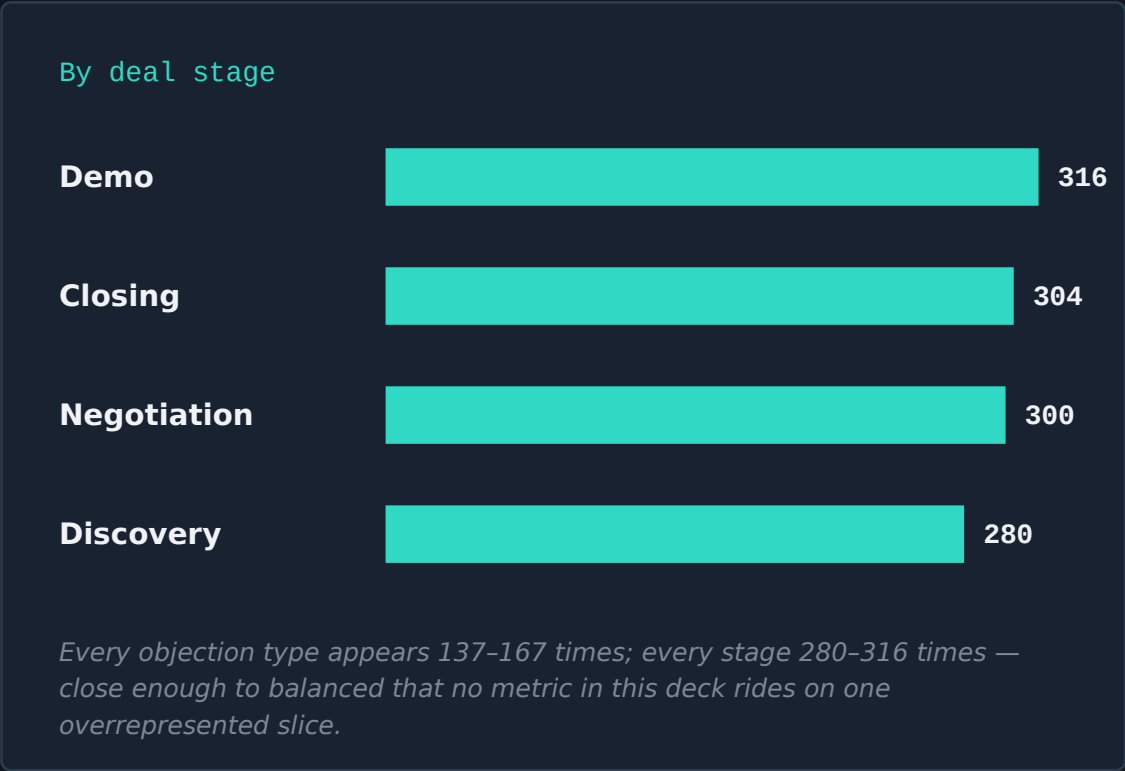
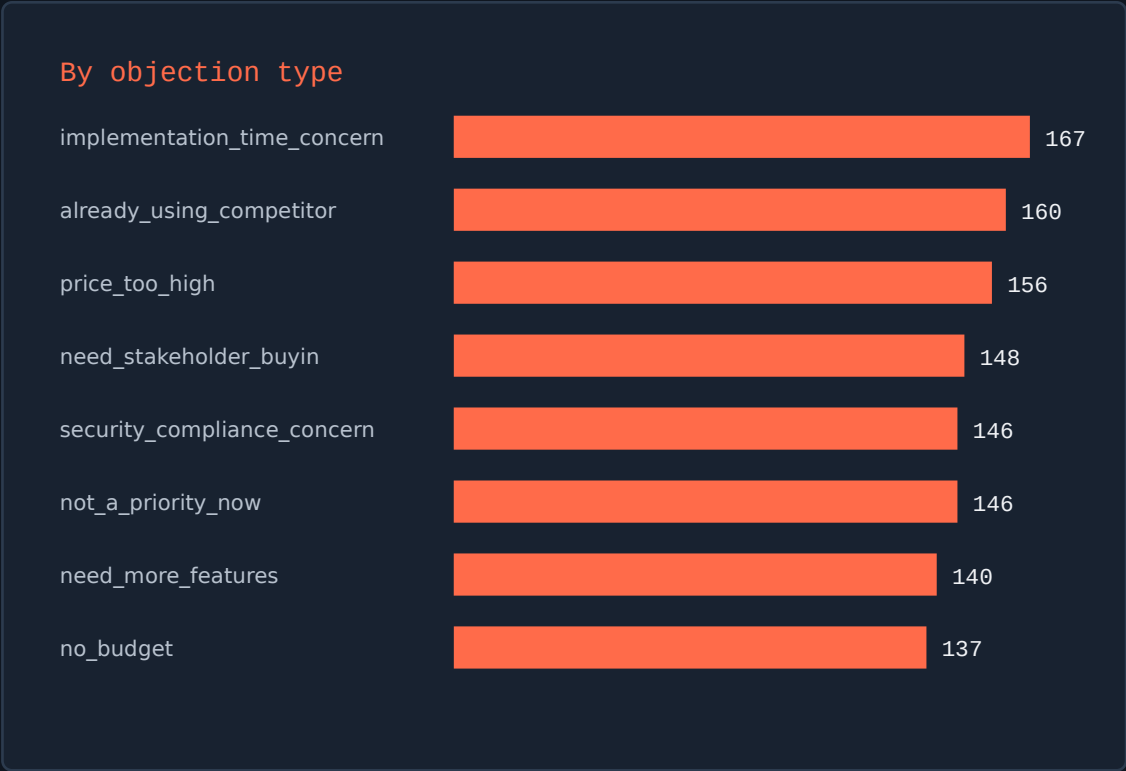
blended win rate

57.7%

of calls used the recommended play

Balanced across objection types and deal stages

1,200 calls, spread evenly enough across 8 objection types and 4 deal stages that no single category dominates the evaluation — real breadth, not a convenient sample.



Two document types, eight objection types, one retrieval target

Every retrieval in this pipeline searches the same 300-document set — a near-even split between short tactical battlecards and longer narrative case studies.



157

battlecards — short, tactical, direct rebuttal scripts



143

case studies — longer, narrative proof points from real deals



30–48

documents per objection type — real depth, real potential for near-duplicates



Why the per-type count matters later

30–48 documents per objection type means several genuinely similar battlecards can exist side by side — exactly the condition that makes exact-document retrieval harder than it needs to be (Slide 28).

DEAL SIZE & OUTCOMES

A realistically skewed book of business

Deal sizes span two orders of magnitude — a shape any real B2B sales corpus would show, and worth knowing before reading a dollar-weighted business-impact number later (Slide 34).



\$7,706

median deal size — the typical deal in this book



\$13,613

mean deal size — pulled well above the median by a long tail



\$297,951

largest deal in the corpus — nearly 39× the median

A right-skewed deal-size distribution like this is exactly why Slide 34 reports a per-deal dollar figure rather than assuming every won deal is worth the average.

THE GOLD EVALUATION SET

Eight queries, one per objection type, hand-labeled

Every objection type gets exactly one canonical query and one human-confirmed "best" document — small by design, and the standard the retriever is measured against in Section 05.

price_too_high	doc 0	no_budget	doc 2
need_more_features	doc 4	already_using_competitor	doc 6
not_a_priority_now	doc 8	need_stakeholder_buyin	doc 10
security_compliance_concern	doc 12	implementation_time_concern	doc 14

Small gold sets are common in early-stage retrieval evaluation — 8 queries is enough to catch a systematic failure, not enough to estimate precision to the decimal point (Slide 28).

A synthetic corpus, disclosed as such



Why synthetic

A generated call corpus and knowledge base remove any dependence on real prospect conversations or proprietary enablement content for a portfolio piece.



What was preserved

A genuine, checkable win-rate association with recommended-play usage, realistic KB duplication, and templated — not naturalistic — transcript phrasing.



What is disclosed

Transcript snippets are structured templates, not natural speech — named explicitly where it changes how a metric should be read (Slide 19).



A methodological choice, stated up front

Every downstream metric in this deck is scoped to this synthetic corpus — not claimed as validated on real sales calls until tested against them directly.



SECTION THREE

Agent 1: Objection Detection

A rule-based classifier that looks strong in aggregate — and hides a total failure mode underneath.

Regex patterns, not a trained classifier — on purpose

The first agent in the pipeline is deliberately simple: keyword and phrase patterns mapped to 8 known objection types, with the exact same output contract a fine-tuned classifier would have.

Example patterns

<code>price_too_high</code>	<code>"too expensive", "price...high"</code>
<code>no_budget</code>	<code>"no budget", "not budgeted"</code>
<code>already_using_competitor</code>	<code>"already use", "competitor"</code>

8 objection types, each with 3-4 regex patterns — the full list ships in `agents/objection_detector.py`.

Why rule-based, deliberately



Zero training data required — ships day one, no labeled corpus needed



Fully auditable — every classification traces to a specific pattern match



Same output contract as a trained classifier — swappable later without touching the rest of the pipeline

KEY INSIGHT

74.4% isn't one number — it's six 100%s and two 0%s

"Six of eight objection types are detected with perfect, 100% accuracy. The remaining two — `implementation_time_concern` and `need_more_features` — are detected correctly 0% of the time. The blended average of 74.4% describes neither case."



Not a borderline miss

This isn't two categories running at 60% instead of 90% — it's two categories where every single call is misclassified as "unclassified."



A single average would have hidden this

Reporting 74.4% alone — without breaking it out by category — would have completely concealed a total failure mode from anyone reading this deck.



Traced to a specific, fixable cause

The root cause is diagnosed on the next slide — this isn't a mystery, it's a known gap with a known fix.

Patterns built for speech, tested on templates

The detector’s patterns were written to catch natural conversational phrasing — exactly what this synthetic corpus’s templated transcripts don’t provide for two of the eight categories.

implementation_time_concern

Pattern needs:

*requires "too long...implement" or
"migration...slow/risky"*

Transcript says:

*"objection raised re: implementation time
concern."*

need_more_features

Pattern needs:

*requires "doesn't have", "missing a feature",
"need...capability"*

Transcript says:

"objection raised re: need more features."

The fix: add a literal "objection raised re:" fallback pattern for evaluation on this corpus, or — for production — replace the rule-based detector with a small fine-tuned classifier trained on real call transcripts.



SECTION FOUR

Agents 2-4: Retrieve, Generate, Score

TF-IDF grounding, a two-tier model escalation, and a deterministic rubric that closes the loop.

Three more agents, chained after detection

Once the objection type is known, three more narrow agents take over — find the right content, turn it into a suggestion, and score the whole exchange.



Agent 2: Retrieve

*TF-IDF search over
300 KB documents*



Agent 3: Generate

LLM call, tiered by
complexity; deterministic
mock when no API key



Agent 4: Score

*a 5-item rubric, deterministic, 100
points total*

Classic TF-IDF, not a neural embedding — by choice



Sparse, lexical matching

A 300-document $\times$ 1,388-term TF-IDF matrix, cosine-similarity ranked — the same technique underlying decades of production search systems.



Filtered by objection type first

Retrieval is scoped to the detected objection type before ranking — the classifier from Agent 1 does real work here, not just labeling.



Interpretable by construction

Every retrieved document's relevance traces to specific shared terms — no embedding space to explain when a rep asks "why this one?"

Implemented with scikit-learn's `TfidfVectorizer` — the fitted vectorizer and document matrix ship together in `tfidf_index.joblib`.

Cheap model for routine cases, strong model for hard ones

The same escalation pattern real production LLM systems use: don't pay for the expensive model on every request, only the ones that need it.

Routine objection

claude-haiku-4-5

Fast, inexpensive — the default tier for the 8 known objection types with clear, direct grounding available.

Zero-cost fallback: a deterministic, template-based mock generator runs the entire pipeline end to end with no API key and no LLM cost — upgrading transparently the moment a key is set.

Complex objection

claude-opus-5

Reserved for objections flagged as multi-part or high-stakes — the `complex\_objection` flag threads through from the pipeline orchestrator.

Every suggestion, at either tier, is prompted to cite the specific enablement document it draws from — grounding is a prompt-level requirement, not an afterthought.

AGENT 4: SCORING

Five weighted checks, zero subjectivity

Every coaching suggestion is scored the same way, by code — not by a second LLM call judging the first one’s output.

Objection correctly identified		25 pts	Agent 1’s output matches the true objection type
Grounded response used		30 pts	the suggestion actually draws on a retrieved document’s title or content
Citation provided		20 pts	the response names its source — "source:" or "doc_id" appears
Response latency acceptable		15 pts	the full pipeline returns in ≤ 3.0 seconds
No hallucinated claim detected		10 pts	no number in the response is absent from every retrieved document
100 points total — the same rubric scores every call, every time, with zero rater drift between evaluations.			

WHY NOT "LLM JUDGING LLM"?

A deliberate architecture choice, not a shortcut



No score drift between runs

A deterministic rubric returns the exact same score for the exact same inputs, every time — an LLM judge can drift run to run on identical input.



Every point is auditable

A manager can trace exactly why a call scored 65/100 — which of the 5 checks passed and which didn't — not just trust a black-box judgment.



Cheaper, faster, at scale

Scoring is plain code, not another API call — free and instant, which matters when every one of 1,200+ calls gets scored.

This mirrors a real pattern from production coaching platforms: a rule-based rubric layer sitting above raw LLM output specifically to remove the score drift an LLM judge would introduce.



SECTION FIVE

Evaluation & Validation

Three separate studies — retrieval, rubric agreement, and outcomes — each read for exactly what it does and doesn't prove.

Three studies, three different questions

Detection was validated against the full call corpus (Section 03). These three studies test the rest of the pipeline — retrieval quality, scoring consistency, and real-world outcomes.



Does retrieval find the right content?

Precision@3 against the 8-query gold set — measured two ways, exact and semantic.



Does the rubric score consistently?

Correlation against a simulated human-rating proxy, across 80 sampled calls.

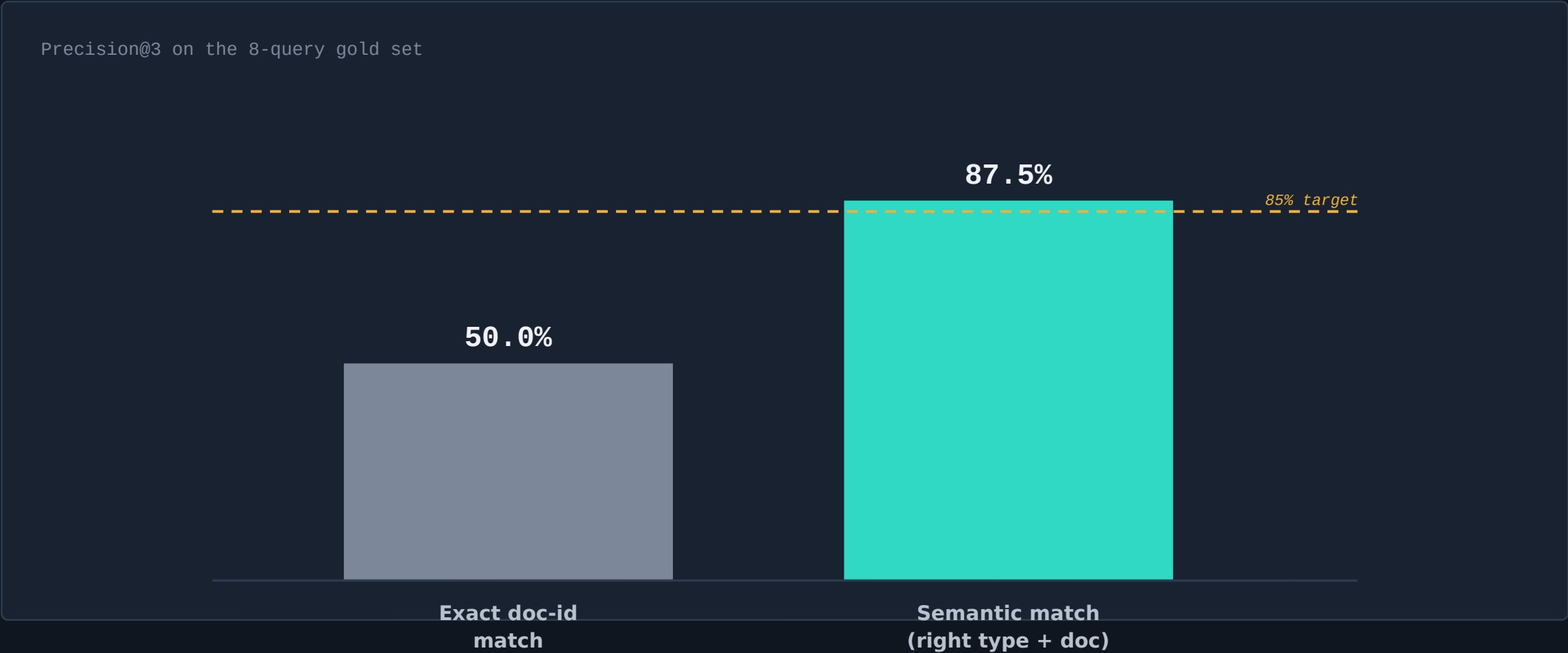


Does the recommended play actually work?

Win rate and expected value, recommended play vs. improvised, across all 1,200 calls.

THE HEADLINE CHART

Precision@3 — exact document match vs. semantic match



THE HONEST RESULT

0.829 "agreement" is with a simulation, not a coach

"The rubric-vs-human correlation of 0.829 clears the 0.80 target — but the "human" score is rubric score plus simulated noise, not a real coach's independent rating. This validates the rubric's internal consistency, not true inter-rater reliability."



Disclosed in the source, not discovered

The evaluation script's own docstring names this limitation — this deck is reporting the caveat, not catching an oversight.



A real study still needs real coaches

A genuine inter-rater reliability study requires independent human ratings on the same calls — named explicitly as required future work (Slide 39).



Still a meaningful internal check

A 0.829 correlation does show the rubric behaves consistently under its own noise model — a legitimate, if narrower, form of validation.

PUTTING THE STUDIES TOGETHER

Where each result stands, honestly

Retrieval precision (semantic)

87.5%

target 85%

Met, on a small but representative 8-query gold set

Rubric-human agreement

0.829

target 0.80

Met against a simulated proxy — real study still needed

Win rate, recommended play

61.3%

vs. 21.1% improvised

A real, checkable association across all 1,200 calls

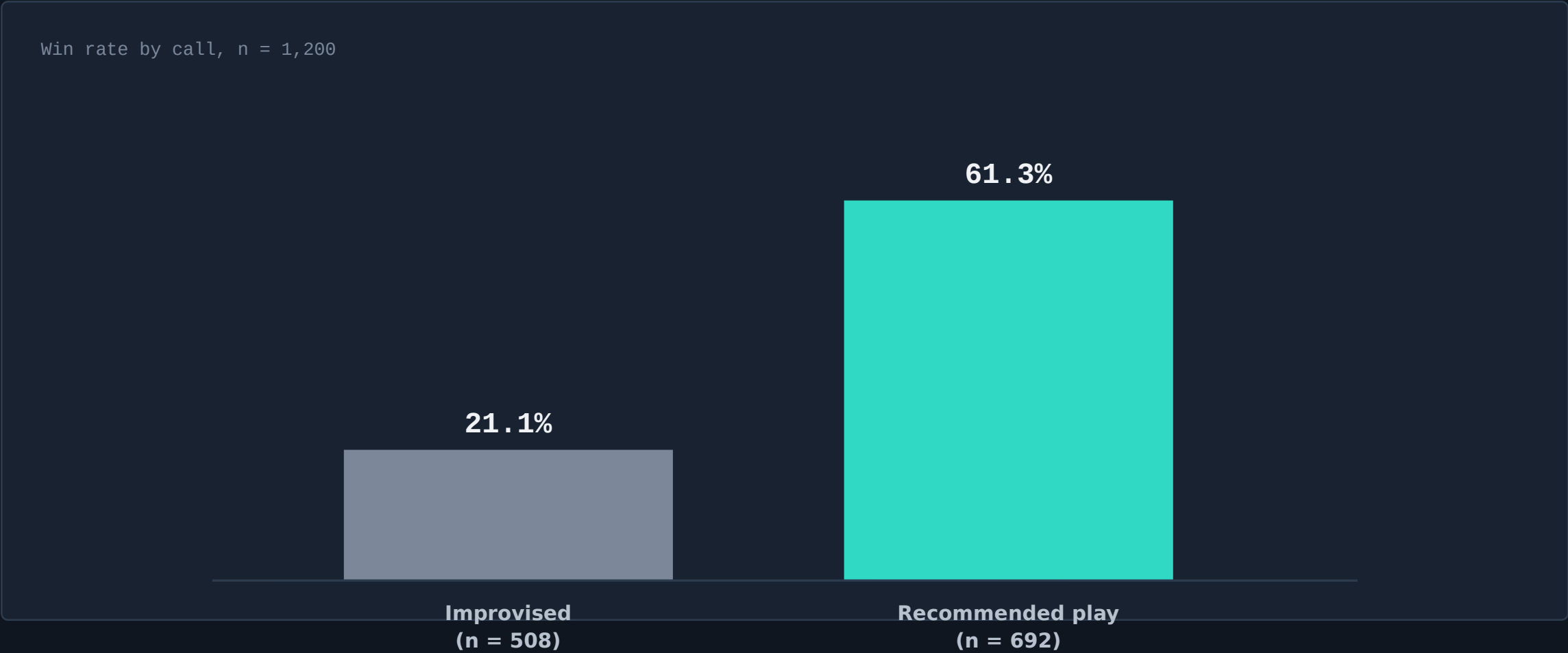


SECTION SIX

Business Impact

A 2.9× win-rate lift is the strongest number in this deck — and the one most worth reading carefully.

Win rate, recommended play vs. improvised



READING THE LIFT HONESTLY

A 2.9× lift is an association, not a randomized trial

“This corpus records which calls used the recommended play and which won — not a controlled experiment that randomly assigned reps to one condition or the other. Stronger reps may both follow the playbook more and close more, for reasons the data alone can’t separate.”



One confound directly checked

Mean deal size is actually slightly lower for recommended-play calls (\$13,221 vs. \$14,390) — the lift isn’t explained by reps cherry-picking easier, smaller deals.



Still real, still actionable

Even read cautiously, a signal this large and this consistent is worth acting on — the honest caveat doesn’t erase the finding, it scopes it.



The rigorous next step

A true causal estimate needs a randomized or quasi-experimental rollout — named explicitly as the validation this number still needs (Slide 39).

WHAT THE LIFT IS WORTH

Expected value per call, not just win rate

Win rate alone doesn't say what a call is worth — multiplying by deal size turns the lift into a number a revenue leader can actually plan around.



Across the 692 calls that used the recommended play, won deals totaled \$5.6M; the 508 improvised calls closed \$1.5M — real dollars behind the abstract percentages.



SECTION SEVEN

Deployment & Responsible Use

The live API, a model card, and the guardrails that keep this a coaching aid, not an autonomous voice in the room.

A FastAPI service, not a dashboard — by design

This system is built to sit behind a call platform, not to be stared at — a lightweight REST endpoint a real-time client calls the moment an objection is detected.

POST /coach

```
{
  "transcript_snippet": "...honestly this feels too
    expensive for our budget...",
  "true_objection_type": null
}
```

200 OK

```
{
  "detected_objection": "price_too_high",
  "retrieved_docs": ["ROI Reframe Battlecard", "..."],
  "coaching_suggestion": "Grounded in \"ROI Reframe..."
```

Also exposed



GET /health — liveness check for the service



Full pipeline runs in under 3 seconds per call, mock or real API



Zero-cost mock mode for demos and CI, upgrades transparently with an API key

Model card summary



Intended use

A real-time coaching aid that detects an objection, retrieves grounded enablement content, and suggests a talking point. Always a suggestion for the rep to use or ignore — never spoken for them.



Limitations

Detection fails completely on 2 of 8 objection types on this corpus (Section 03). Rubric-human agreement is validated against a simulated proxy, not real coaches. Win-rate lift is correlational.



Next steps

Fix the two failing detection patterns, run a real inter-rater study with human coaches, and design a randomized rollout to estimate the win-rate lift causally (Slide 39).

Guardrails for a system that whispers, never speaks



A suggestion engine, never an autonomous voice

The system never speaks to a prospect and never auto-sends anything — every output is a suggestion a human rep chooses to use, or not.



Grounding and citation are enforced, not optional

The rubric docks points for an ungrounded or uncited response — the same standard the system holds itself to is visible in every score.



Known failure modes are disclosed, not hidden

The two objection types the detector currently misses (Section 03) are named plainly — a rep team using this tool should know where it goes quiet.



Escalation, not blanket high-cost inference

Only objections flagged as complex reach the stronger, more expensive model — keeping the system's cost and latency proportionate to the task.

From reference implementation to a validated product



Fix the detection blind spot

Add fallback patterns or a small fine-tuned classifier for the two objection types the rule-based detector currently misses entirely.



Run a real inter-rater study

Replace the simulated human-rating proxy with independent ratings from real sales coaches on the same call sample.



Design a causal rollout

A randomized or staged rollout across comparable reps to estimate the win-rate lift as a genuine causal effect, not just an association.



Expand the gold evaluation set

Grow retrieval evaluation beyond 8 queries for a tighter precision estimate as the knowledge base grows.

KEY TAKEAWAYS

A real lift, and three metrics reported with the caveats they need



**2.9× win-rate lift,
recommended play**



**74.4% detection hides a
100%/0% split**



**87.5% semantic retrieval
precision**



**0.829 agreement, vs. a
simulated proxy**

Skills demonstrated

Agentic pipeline design · retrieval-augmented generation · deterministic rubric evaluation · LLM cost-tiered routing · honest multi-study reporting

Anthony Chinedu Echem

Machine Learning Engineer & Data Scientist



Portfolio contact available on request

